



GitHub Actions Hands-on Labs

From your first workflow to CI/CD: tests, matrix builds, PR checks, protected deploys, releases, reusable workflows

Self-learning, step-by-step deck for early-career developers | FIL Gurgaon | Python projects

How to use this deck (read once)

DO

Dark box = type this in a terminal, a file or a GitHub field. Copy exactly.

VERIFY

Green box = what you MUST see before moving on - your proof the step worked.

IF IT FAILS

Amber box = the most common error for this step and its fix.

WHY

Blue box = the idea behind the step - what you will use at work.

Conventions

- `<you>` = your GitHub username. Replace it everywhere.
- `$` = laptop terminal (Windows: Git Bash). Do not type the `$`.
- GitHub clicks: **Repo** → **Settings** → **Environments**
- Workflow files live in `.github/workflows/` - exact folder name, with the dot.
- Stuck? The finished file for every lab is in `solution/labN/`.
- From Lab 4 on, every change goes through a pull request (the 'PR routine' slide).

i Golden rule

- Never skip a VERIFY. A red run 3 steps later is far harder to debug than a missing check now.

What you will build today

1

Lab 1 Hello workflow

events, jobs, steps, logs, manual run with input

2

Lab 2 CI for Python

lint + test, annotations, artifacts, cache, badge

3

Lab 3 Jobs & matrix

needs graph, 3 Python versions, wheel artifact

4

Lab 4 Pull-request checks

rulesets, blocked merges, concurrency

5

Lab 5 Delivery

protected environment, secret, approval, release

6

Lab 6 Reuse

reusable workflow, schedule, Dependabot

i One project for every lab

- Expense Claims API - a real FastAPI service (JWT login, role-based expense list) with 13 tests, Ruff lint rules, a wheel build, a deploy script and a smoke test - all in the kit.
- Everything runs on GitHub-hosted runners: free for public repositories, nothing to install except Git and Python.

Agenda and timing (one full day)

Part	What you do	Time
Concepts	CI/CD, building blocks, workflow anatomy, YAML, runners, actions, expressions	45 min
Lab 1 Hello workflow (5 steps)	First workflow, read logs, manual run with input, broken YAML	40 min
Lab 2 CI for Python (6 steps)	Lint + test, annotations, test-report artifact, cache, badge	60 min
Lab 3 Jobs & matrix (4 steps)	lint → test x3 Pythons → build; a version-specific failure	50 min
Lab 4 PR checks (5 steps)	Ruleset, PR routine, blocked merge, cancel outdated runs	50 min
Lab 5 Delivery (4 steps)	staging environment + secret + approval, deploy + smoke, tag release	60 min
Lab 6 Reuse (4 steps)	Reusable workflow, ruleset update, nightly schedule, Dependabot	45 min
Wrap-up	Security checklist, quiz, clean-up	20 min

i Pace

- Breaks after Lab 2 and after Lab 4. Runs take 30 s - 2 min: read the WHY box while you wait. Fast finishers: STRETCH steps or help a neighbour.

Before class: laptop checklist (10 minutes)

laptop terminal

```
$ git --version           # 2.28 or newer
$ python3 --version       # 3.11, 3.12 or 3.13
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"

# Can you push to GitHub? (prints refs or asks to log in)
$ git ls-remote https://github.com/actions/checkout | head -2
```

✓ You are ready when

- Each command prints a version - no 'command not found'.
- ls-remote prints two lines starting with a commit hash.
- You can sign in at github.com with your own account.
- Editor: VS Code (+ optional 'GitHub Actions' extension by GitHub - it validates workflow YAML as you type).

i Why public repositories?

- GitHub-hosted runner minutes are free for public repos, and environments with required reviewers work on every plan for public repos. The lab project contains no real secrets.

! Common blockers

- Corporate proxy: git config --global http.proxy http://<proxy>:<port>
- Windows: install Git for Windows (includes Git Bash) and Python from python.org (tick 'Add to PATH').
- Push asks for a password: GitHub needs a browser login (Git Credential Manager) or a token - see Step 1.1.

Your lab kit: what is in the folder

```
FIL-GitHub-Actions-Labs-Kit/
├── README.md
├── TROUBLESHOOTING.md
├── starter/expense-api/
└── solution/
    ├── lab1/hello.yml
    ├── lab2/ci.yml
    ├── lab3/ci.yml
    ├── lab4/ci.yml
    ├── lab5/ci.yml
    ├── lab5/release.yml
    └── lab6/
```

quick start + verified versions
symptom → cause → fix
the project you push to GitHub
finished workflow files per lab

1 job: lint + test + report
lint → test matrix → build
+ PR triggers + concurrency
+ deploy-staging job
tag → GitHub Release
python-quality.yml (reusable),
ci.yml, nightly.yml, dependabot.yml

folder tree

i Starter vs solution

- You type every workflow yourself - typing YAML is how indentation rules stick.
- solution/ files are the reference: compare with yours when a run goes red and you cannot see why.
- Every solution file passed actionlint, and every command inside was executed before class (see README).

✓ Check

- `ls FIL-GitHub-Actions-Labs-Kit/solution` shows lab1 ... lab6.

Concepts The 45 minutes that make the labs easy

CI/CD | building blocks | workflow anatomy | YAML | runners | actions & versions | expressions | Jenkins mapping

Why CI/CD? The problem it solves

Without CI: 'works on my machine'

- Tests run only when someone remembers.
- Broken code reaches main; the next person finds it.
- Builds and deploys are manual and differ per person.
- Nobody knows which version is on staging.

With CI/CD: every push is checked the same way

- CI: every push/PR is linted, tested, built automatically.
- Red check = merge blocked before anyone is affected.
- CD: the same artifact is deployed the same way, with approvals.
- History shows who changed what, when, and what ran.

The loop you will automate today



i Continuous Integration vs Delivery

- CI = merge small changes often, each one verified automatically. CD = every verified change is ready to deploy; going live may need an approval (Lab 5).

The 6 building blocks of GitHub Actions

1

Event

Something that happens: push, pull_request, workflow_dispatch (button), schedule, a tag.

2

Workflow

A YAML file in .github/workflows/. Says WHEN (on:) and WHAT (jobs:).

3

Job

A group of steps on ONE fresh runner. Jobs run in parallel unless 'needs:' orders them.

4

Step

One command (run:) or one action (uses:). Steps of a job run in order and share files.

5

Runner

The VM that runs a job. ubuntu-latest = GitHub-hosted Linux, created new for every job.

6

Action

A reusable step from a repo, e.g. actions/checkout@v7. Like a library function.

i Remember this sentence

- An EVENT triggers a WORKFLOW; the workflow's JOBS run on RUNNERS; each job runs STEPS; a step is a shell command or an ACTION.

Anatomy of a workflow file

.github/workflows/ci.yml

```
name: CI                                # shown in the Actions tab

on:                                     # EVENTS that start it
  push:
  pull_request:
  workflow_dispatch:                   # 'Run workflow' button

permissions:                           # what the job token may do
  contents: read

jobs:
  quality:                             # job id (your choice)
    runs-on: ubuntu-latest             # RUNNER
    steps:
      - uses: actions/checkout@v7      # an ACTION
      - uses: actions/setup-python@v7
        with:                           # inputs of the action
          python-version: "3.13"
      - name: Test                      # a shell STEP
        run: pytest -q
```

i Read it top-down

- **on:** - when does it run?
- **permissions:** - least privilege for the automatic GITHUB_TOKEN.
- **jobs:** - map of job-id → job. Each job gets a brand-new VM.
- **steps:** - list (each item starts with -). **uses:** = action, **run:** = shell.
- **with:** - inputs for an action; **env:** - environment variables for a step.
- The runner starts EMPTY: without actions/checkout your code is not there (you will prove it in Lab 1).

YAML in 5 rules (90% of beginner errors)

yaml rules

```
# 1. Indent with SPACES (2 per level). Never tabs.
jobs:
  build:
    runs-on: ubuntu-latest

# 2. 'key: value' needs a space after the colon
name: CI           # ok      name:CI    # wrong

# 3. A list item starts with '- '
steps:
  - run: echo one
  - run: echo two

# 4. Multi-line shell script: use |
run: |
  echo line 1
  echo line 2

# 5. Quote values that look like numbers/booleans
python-version: "3.10"  # unquoted 3.10 becomes 3.1 !
```

i Why rule 5 matters

- YAML reads 3.10 as the number 3.1, and GitHub then installs Python 3.1 (or fails). Always quote versions: "3.13".

! What a YAML error looks like

- The run appears immediately with a red X and 'Invalid workflow file' plus the line number. Nothing ran. Fix the line, push again. You will cause one on purpose in Step 1.5.

Tip

- VS Code shows indentation guides. Turn on 'Render Whitespace' to spot tabs.

Runners, actions and versions

GitHub-hosted runners

- `ubuntu-latest` (used today), `windows-latest`, `macos-latest`.
- A fresh VM per job; deleted afterwards. Files do not survive between jobs - use artifacts (Lab 3).
- Pre-installed: git, Python, gh CLI, Docker, curl...
- Free for public repos; private repos use the account's minutes.
- Self-hosted runners: your own machines (company networks). Same YAML, runs-on: self-hosted.

Actions = reusable steps

- `uses: owner/repo@ref` - e.g. `actions/checkout@v7`
- Find them at github.com/marketplace?type=actions. Prefer 'actions/*' (by GitHub) and verified creators.
- `@v7` = major tag (gets fixes). Strictest: pin a full commit SHA.
- Versions used today (latest majors): `checkout v7`, `setup-python v7`, `upload-artifact v7`, `download-artifact v8`.

```
- uses: actions/checkout@v7          # major tag - convenient
- uses: actions/checkout@v7.0.1      # exact tag - can still be moved
- uses: actions/checkout@<40-character-commit-sha> # v7.0.1 # SHA pin - immutable, most secure
```

three ways to pin

i At work

- Security teams often require SHA pins for third-party actions, kept current by Dependabot (Lab 6).

Expressions, contexts and variables

cheat sheet

```
${{ github.ref_name }}      # branch or tag name          (github context)
${{ github.event_name }}    # push, pull_request, ...
${{ inputs.greeting }}      # workflow_dispatch / workflow_call input
${{ matrix.python-version }} # current matrix value      (Lab 3)
${{ secrets.JWT_SECRET }}   # encrypted secret           (Lab 5)
${{ github.token }}         # automatic GITHUB_TOKEN         (Lab 5)

if: always()                # run this step even after a failure
if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'

# Inside run: the same data as ENV variables (no ${{ }} needed)
run: echo "$GITHUB_REF_NAME $GITHUB_SHA $RUNNER_OS"
```

i Two syntaxes, one idea

- `${{ ... }}` is evaluated by GitHub BEFORE the step runs.
- `$NAME` is read by the shell WHILE the step runs.
- Default env vars (`GITHUB_*`, `RUNNER_*`) are always there - you will print them in Lab 1.

! Security rule: never put `${{ }}` user input straight into run:

- Bad: `run: echo "${{ inputs.greeting }}"` - the text is pasted into the script before it runs (script injection).
- Good: `env: GREETING: ${{ inputs.greeting }}` then `run: echo "$GREETING"`
- The same for PR titles, branch names, issue text - anything a user can type.

Coming from Jenkins? Same ideas, new names

Jenkins	GitHub Actions
Jenkinsfile in the repo	<code>.github/workflows/*.yaml</code> (many files allowed)
Controller + agents you operate	GitHub-hosted runners (or self-hosted)
<code>stage('Lint') { steps { sh ... } }</code>	<code>jobs: lint: steps: - run: ...</code>
Stages in one workspace	Jobs on separate VMs - share files via artifacts
<code>post { always { junit ... } }</code>	<code>if: always()</code> on a step
Credentials + withCredentials	Secrets (repo / environment) + <code>env:</code>
<code>input step (approval)</code>	Environment with required reviewers
Shared Library @Library	Reusable workflow (<code>workflow_call</code>) / composite action
pollSCM / webhook	Events are native: <code>push</code> , <code>pull_request</code> , <code>schedule</code>
archiveArtifacts	<code>actions/upload-artifact</code>

i Which one at work?

- Company Jenkins (FIL shared server) for existing pipelines; GitHub Actions when code and CI live on GitHub. The design skills are identical.

Lab 1 Your first workflow

5 steps | 40 min | Push the project, write hello.yml, read the logs, run it by button with an input, break the YAML on purpose.

Step 1.1 Create the repo and push the project

LAB 1 · STEP 1 of 5

1 **github.com** → + → **New repository** name **expense-api**, Public, NO README/.gitignore/licence → Create repository

2 Copy the starter, run its checks once, push:

```
laptop terminal
$ cp -r FIL-GitHub-Actions-Labs-Kit/starter/expense-api ~/expense-api
$ cd ~/expense-api
$ python3 -m venv .venv && source .venv/bin/activate
# Git Bash on Windows: source .venv/Scripts/activate
(.venv) $ pip install -r requirements-dev.txt
(.venv) $ ruff check app tests scripts && pytest -q

$ git init -b main && git add . && git commit -m "Expense API starter"
$ git remote add origin https://github.com/<you>/expense-api.git
$ git push -u origin main
```

✓ You should see

- 'All checks passed!' and '13 passed' locally - CI will run exactly these commands.
- Repo page on GitHub lists app/, scripts/, seed/, tests/, pyproject.toml. No .venv.
- Actions tab says 'Get started with GitHub Actions' (no workflows yet).

! If the push asks for a password

- GitHub refuses account passwords on the command line.
- Windows/macOS: Git Credential Manager opens a browser - sign in.
- Else: github.com → Settings → Developer settings → Personal access tokens → Fine-grained → this repo, 'Contents: Read and write' AND 'Workflows: Read and write'. Paste it as the password.

i Why 'Workflows' permission?

- Pushing files under .github/workflows/ needs the extra 'workflow' scope - a classic first-day error: 'refusing to allow ... to create or update workflow'.

Step 1.2 Write hello.yml

LAB 1 · STEP 2 of 5

hello.yml (part 1)

```
name: Hello Actions
on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      greeting:
        description: "Who should we greet?"
        required: true
        default: "FIL Gurgaon"
permissions:
  contents: read
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Say hello
        env:
          GREETING: ${ inputs.greeting || 'push event' }
        run: echo "Hello, $GREETING!"
```

part 2 (same file)

```
- name: Show where we are running
  run: |
    echo "Event:      $GITHUB_EVENT_NAME"
    echo "Repository: $GITHUB_REPOSITORY"
    echo "Branch:      $GITHUB_REF_NAME"
    echo "Commit:      $GITHUB_SHA"
    echo "Runner OS:   $RUNNER_OS"
    python3 --version
- name: Workspace BEFORE checkout
  run: ls -la
- name: Check out the repository
  uses: actions/checkout@v7
- name: Workspace AFTER checkout
  run: ls -la
- name: Write a job summary
  run: |
    echo "### Hello from run #$GITHUB_RUN_NUMBER" >>
"$GITHUB_STEP_SUMMARY"
    echo "Triggered by **$GITHUB_EVENT_NAME**" >> "$GITHUB_STEP_SUMMARY"
```

```
$ mkdir -p .github/workflows # then create hello.yml in it
$ git add .github && git commit -m "Add hello workflow" && git push
```

Step 1.3 Watch the run and read the log

1	Repo → Actions tab	left list shows 'Hello Actions'
2	Click the newest run	title = your commit message
3	Click job 'hello'	steps appear, each expandable
4	Expand 'Show where we are running'	Event: push, Branch: main, Runner OS: Linux
5	Expand 'Workspace BEFORE checkout'	only . and .. - the VM is empty!
6	Expand 'Workspace AFTER checkout'	app, scripts, seed, tests, .git ...
7	Back to run page (Summary)	'Hello from run #1' under the job summary

✓ You should see

- Green tick on the run and on the commit (repo home page → next to the commit message).
- 'Set up job' shows the runner image (Ubuntu) and the GITHUB_TOKEN permissions: Contents: read.
- The python3 --version line prints a Python 3.x version (pre-installed).

i The 3 lessons of this run

- A push to main was the EVENT - you clicked nothing.
- The runner is a brand-new empty VM: checkout is always your first real step.
- GITHUB_* variables tell a step where and why it runs - useful in scripts and debugging.

! No run appears?

- File not in .github/workflows/ (check spelling, the dot), or pushed to another branch than main.

Step 1.4 Run it by button with your own input

LAB 1 • STEP 4 of 5

1	Actions → Hello Actions	(left list)
2	Run workflow ▾ (right side)	Branch: main
3	Who should we greet?	Asha from FIL
4	Run workflow (green)	a new run appears in a few seconds

✓ You should see

- Run label 'Manually run by <you>'.
- 'Say hello' log: Hello, Asha from FIL!
- 'Show where we are running': Event: workflow_dispatch.
- Job summary: 'Triggered by workflow_dispatch'.

Try (1 min)

- Type a greeting with quotes and \$ signs, e.g. `" ; echo hacked; "` - it is printed as plain text, never executed. That is the env: pattern protecting you.

i workflow_dispatch

- Manual trigger with typed inputs - great for 'deploy now', 'rerun nightly', demos.
- The button only appears when the workflow file is on the default branch (main).
- Inputs: string, choice, boolean, number, environment.

! No 'Run workflow' button?

- The workflow lacks workflow_dispatch:, or that version is not on main yet. Refresh after the push.

Step 1.5 Break the YAML on purpose, then fix it

LAB 1 • STEP 5 of 5

laptop terminal / editor

```
# In hello.yml change the indentation of runs-on (2 spaces less):
hello:
  runs-on: ubuntu-latest      # <- now at the same level as 'hello:'

$ git commit -am "break yaml" && git push

# Fix: put the 2 spaces back (runs-on under hello:), or undo the commit
$ git revert --no-edit HEAD && git push
```

✓ You should see

- Run with a red X almost instantly, 0 jobs executed.
- Message 'Invalid workflow file: .github/workflows/hello.yml' and 'You have an error in your yaml syntax on line ~19' (the line of steps:, now orphaned).
- After the revert: the next run is green again.

i What you learned

- GitHub validates the YAML before anything runs - read the line number it gives.
- Indentation IS structure in YAML: 2 spaces moved runs-on out of the job.
- git revert = safe undo that keeps history (no force-push).
- Lab 1 done: compare your file with solution/lab1/hello.yml.

Lab 2 Continuous Integration for Python

6 steps | 60 min | Lint + test on every push, error annotations on the exact line, a downloadable test report, pip cache and a status badge.

Step 2.1 Write ci.yml (one job, 6 steps)

LAB 2 • STEP 1 of 6

.github/workflows/ci.yml

```
name: CI
on:
  push:
  pull_request:
  workflow_dispatch:
permissions:
  contents: read
jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - name: Check out the code
        uses: actions/checkout@v7
      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.13"
          cache: pip
          cache-dependency-path: requirements*.txt
      - name: Install dependencies
        run: pip install -r requirements-dev.txt
      - name: Lint (ruff)
        run: ruff check app tests scripts --output-format=github
      - name: Test (pytest)
        run: pytest -q --junitxml=reports/junit.xml
      - name: Upload test report
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: test-report
          path: reports/junit.xml
```

i Line by line

- setup-python: installs exactly 3.13 (not whatever the VM has). cache: pip re-uses downloads; the key changes when requirements*.txt change.
- --output-format=github: ruff prints errors as GitHub annotations (shown on the code line).
- --junitxml: machine-readable test report.
- if: always(): upload the report even when tests failed - that is when you need it.
- No 'branches:' filter yet: runs on every branch. Lab 4 refines it.

```
$ git add .github && git commit -m "Add CI" && git push
```

✓ Verify

- Your file = solution/lab2/ci.yml.

Step 2.2 Read the green run: logs, artifact, timing

job 'quality' log (shortened)

```
Run ruff check app tests scripts --output-format=github
All checks passed!
Run pytest -q --junitxml=reports/junit.xml
..... [100%]
- generated xml file: /home/runner/work/expense-api/expense-api/reports/junit.xml -
13 passed in 11.2s
```

✓ You should see

- Actions now lists 2 workflows: CI and Hello Actions (both ran for this push).
- CI run green; job 'quality' shows 6 named steps + Set up job / Post steps / Complete job.
- Run summary page → 'Artifacts' section → test-report. Click to download a zip containing junit.xml.
- Each step shows its duration on the right - the install step is the slowest.

! If it fails

- 'No file matched to [*/requirements*.txt]' style cache error → the cache path does not match your files. 'ruff: command not found' → Install step missing or misspelt file name.

i Where are the files?

- On the runner the repo is checked out to /home/runner/work/<repo>/<repo>.
- When the job ends the VM is destroyed. Only uploaded artifacts (kept 90 days by default, configurable) and logs remain.
- That is why reports and packages must be uploaded explicitly.

Step 2.3 Break the lint: see the annotation on your line

LAB 2 • STEP 3 of 6

laptop terminal

```
$ echo "import os" >> app/__init__.py
$ git commit -am "break lint" && git push

# after looking at the red run:
$ git revert --no-edit HEAD && git push
```

raw log

```
::error title=ruff (F401),file=.../app/__init__.py,line=1,col=8,...
::app/__init__.py:1:8: F401 `os` imported but unused
Error: Process completed with exit code 1.
```

✓ You should see

- CI red at 'Lint (ruff)'; 'Test (pytest)' shows as skipped; 'Upload test report' still ran (if: always) but warns no file found.
- Run summary → Annotations: 1 error - F401 `os` imported but unused, app/__init__.py line 1.
- Commit page (click the commit hash): the error is displayed next to that line.
- After the revert: green.

i Fail fast, point exactly

- A failing step stops the job; later steps are skipped unless they use if: always().
- Annotations bring the error to the code, also in pull-request 'Files changed' - reviewers see it without opening logs.
- Exit code ≠ 0 is the only thing that turns a step red.

Notice

- Hello Actions ran too (push to main) and stayed green - workflows are independent.

Step 2.4 Break a test: read the report artifact

LAB 2 • STEP 4 of 6

laptop terminal

```
# app/main.py: "X-Frame-Options": "DENY" -> "SAMEORIGIN"
$ sed -i.bak 's/"DENY"/"SAMEORIGIN"/' app/main.py && rm app/main.py.bak
$ git commit -am "break test" && git push

# after inspecting:
$ git revert --no-edit HEAD && git push
```

✓ You should see

- Lint green, 'Test (pytest)' red: FAILED tests/test_expenses.py::test_security_headers_present ... 1 failed, 12 passed.
- 'Upload test report' is green (if: always). Download test-report → junit.xml contains <failure ...> for test_security_headers_present.
- After the revert: all green.

Discuss (2 min)

- Lint passed but a security header was weakened. Which check protected you? What would happen without CI?

i Why upload the report?

- Logs are text for humans; JUnit XML feeds dashboards, test-reporting actions and IDEs.
- In big projects you download the report instead of scrolling 10,000 log lines.
- Stretch idea: a test-reporter action can turn junit.xml into a summary table (not needed today).

Step 2.5 Prove the pip cache works

LAB 2 • STEP 5 of 6

1	Open the FIRST CI run	Set up Python log: no pip cache found yet
2	Same run → 'Post Set up Python'	saves the cache ('Cache saved...')
3	Open the LATEST CI run	Set up Python log: 'Cache restored from key: setup-python-...-pip-...'
4	Compare 'Install dependencies'	duration is shorter on the cached run
5	Repo → Actions → Caches (left menu)	one entry setup-python-Linux-...-pip-...

✓ You should see

- The words 'Cache restored' in a later run, and a cache entry in Actions → Caches. (Exact seconds vary - what matters is restored vs not found.)

i How the cache key works

- Key = OS + Python + pip + hash of requirements*.txt.
- Change a pinned version → new hash → cache miss once → new cache saved.
- Caches are a speed-up, never a source of truth: CI must also pass with an empty cache.
- Caches unused for 7 days are deleted automatically.

Step 2.6 Add a status badge to the README

LAB 2 · STEP 6 of 6

```
# first line of README.md:  
![CI] (https://github.com/<you>/expense-api/actions/workflows/ci.yml/badge.svg)  
  
$ git commit -am "README: CI badge" && git push
```

README.md

Or let GitHub write it

- Actions → CI → '...' (top right) → Create status badge → Copy status badge Markdown.

✓ You should see

- Repo home page shows a 'CI | passing' badge above the README text.
- Clicking the badge opens the CI workflow runs.

i Lab 2 checkpoint

- Every push is linted and tested automatically.
- Errors appear on the exact line (annotations).
- Reports survive the VM (artifacts).
- Repeat runs are faster (cache).
- Everyone sees the state (badge).
- Compare with solution/lab2/ci.yml. Take a 10-minute break.

Lab 3 Jobs, dependencies and a test matrix

4 steps | 50 min | lint → test on Python 3.11 / 3.12 / 3.13 → build a wheel artifact. Then watch a failure that happens on one Python version only.

Step 3.1 Split the job into lint → test → build

LAB 3 · STEP 1 of 4

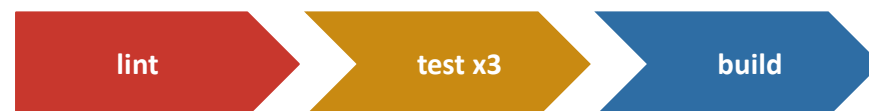
```
ci.yml - jobs (lab3)

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"
          cache: pip
          cache-dependency-path: requirements*.txt
      - run: pip install -r requirements-dev.txt
      - name: Lint (ruff)
        run: ruff check app tests scripts --output-format=github

  test:
    # full version with matrix on the next slide
    needs: lint # starts only after lint is green
    ...

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"
      - name: Build wheel
        run: |
          pip wheel . --no-deps -w dist
          ls -l dist
      - name: Upload wheel
        uses: actions/upload-artifact@v7
        with:
          name: expense-api-wheel
          path: dist/*.whl
          retention-days: 7
```

The job graph you will see



i needs: and the job graph

- Without needs: all jobs start at the same time (parallel).
- needs: lint = wait for lint and stop if it fails - cheapest check first.
- Each job is a NEW VM: every job repeats checkout + setup-python. That is normal.
- Jobs share files only through artifacts (the wheel is uploaded here, downloaded in Lab 5).

Do

- Replace the jobs: section of ci.yml with the lab3 version (type it - the test job is on the next slide), then push.

Step 3.2 Test on 3 Python versions with a matrix

```
test:
  needs: lint
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false          # let every version finish
    matrix:
      python-version: ["3.11", "3.12", "3.13"]
  name: test (py${{ matrix.python-version }})
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-python@v7
      with:
        python-version: ${{ matrix.python-version }}
        cache: pip
        cache-dependency-path: requirements*.txt
    - run: pip install -r requirements-dev.txt
    - name: Test (pytest)
      run: pytest -q --junitxml=reports/junit.xml
    - name: Upload test report
      if: always()
      uses: actions/upload-artifact@v7
      with:
        name: test-report-py${{ matrix.python-version }}
        path: reports/junit.xml
```

the test job

✓ You should see

- Run graph: lint → 3 boxes test (py3.11), test (py3.12), test (py3.13) → build.
- All green; each test job prints '13 passed'.
- Artifacts: test-report-py3.11, -py3.12, -py3.13 and expense-api-wheel.

i Matrix = one definition, N jobs

- GitHub creates one job per value and runs them in parallel.
- Artifact names must be unique per job - hence -py\${{ matrix.python-version }}.
- Quotes again: "3.11" not 3.11.

```
$ git commit -am "CI: jobs + matrix" && git push
```

Step 3.3 A failure on ONE Python version only

LAB 3 · STEP 3 of 4

laptop terminal / editor

```
# tests/test_batched.py    (itertools.batched exists only in Python 3.12+)
import itertools

def test_batched():
    assert list(itertools.batched("abcd", 2)) == [("a", "b"), ("c", "d")]

$ git add tests/test_batched.py && git commit -m "use batched" && git push

# after inspecting: remove it again
$ git rm tests/test_batched.py && git commit -m "drop batched" && git push
```

✓ You should see

- lint green (the code is valid Python).
- test (py3.11) RED: AttributeError: module 'itertools' has no attribute 'batched' - 1 failed, 13 passed.
- test (py3.12) and (py3.13) GREEN: 14 passed - they finished because fail-fast: false.
- build SKIPPED (needs: test failed). After the git rm: all green again.

i Why a matrix pays off

- Your laptop had 3.13 - the bug was invisible locally.
- pyproject.toml promises requires-python >= 3.11, so 3.11 must be tested.
- Real fix options: raise requires-python to 3.12, or avoid the 3.12-only API.
- fail-fast: true (the default) would have cancelled the other legs - quicker, but you learn less.

Step 3.4 Download and inspect the wheel artifact

LAB 3 • STEP 4 of 4

1	Latest green CI run → Summary	scroll to 'Artifacts'
2	Click expense-api-wheel	a .zip downloads
3	Unzip it	expense_api-1.0.0-py3-none-any.whl
4	Optional: look inside	python3 -m zipfile -l expense_api-1.0.0-py3-none-any.whl

✓ You should see

- 4 artifacts on the run: expense-api-wheel + 3 test reports; the wheel shows 'expires in 7 days' style retention.
- zipfile -l lists app/__init__.py, app/main.py ... and expense_api-1.0.0.dist-info/ - no tests.

i Build once, deploy that file

- The wheel built in this job is THE package. Lab 5 deploys exactly this file (download-artifact) instead of rebuilding.
- retention-days: 7 keeps storage small; test reports use the repo default.
- Lab 3 done: compare with solution/lab3/ci.yml.

Lab 4 Protect main with pull-request checks

5 steps | 50 min | Smarter triggers, cancel outdated runs, a branch ruleset that blocks red merges - the way real teams work.

Step 4.1 Smarter triggers + cancel outdated runs

LAB 4 • STEP 1 of 5

ci.yml - new on: + concurrency

```
on:
  push:
    branches: [main]          # feature branches: covered by PR
  pull_request:
    branches: [main]
  workflow_dispatch:

concurrency:
  # newer run cancels older (same ref)
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

```
$ git commit -am "CI: PR triggers" && git push  # last direct push!
```

✓ You should see

- CI runs for this push to main and is green. File equals solution/lab4/ci.yml.

i Why change the triggers?

- Lab 2/3 used 'push' on every branch + 'pull_request': a PR branch ran CI twice.
- Now: PRs into main run CI on the PR; main runs after merge.
- github.ref differs per branch/PR (refs/pull/7/merge), so only runs of the SAME PR cancel each other.
- Avoid 'paths' filters on required checks: a skipped workflow never reports and the PR waits forever.

Step 4.2 Create a branch ruleset for main

LAB 4 • STEP 2 of 5

1	Repo → Settings → Rules → Rulesets	New ruleset → New branch ruleset
2	Ruleset Name	protect-main
3	Enforcement status	Active
4	Target branches → Add target	Include default branch
5	Tick: Require a pull request before merging	Required approvals: 0 (you work alone today)
6	Tick: Require status checks to pass	+ Add checks: lint, test (py3.11), test (py3.12), test (py3.13), build
7	Tick: Block force pushes	(already ticked)
8	Create	

```
laptop terminal
# Prove it: try a direct push
$ echo "Owner: <you>" >> README.md && git commit -am "direct push" && git push
# Move that commit to a branch and reset local main:
$ git switch -c docs/owner && git branch -f main origin/main
```

✓ You should see

- The push is REJECTED: 'GH013: Repository rule violations found for refs/heads/main' - changes must be made through a pull request / required status checks.

! Check names not offered?

- Names appear only after they ran in the last days. Run CI once (Actions → CI → Run workflow), then type the names again - exactly as shown on the run page.

i Rulesets

- Rules for branches/tags, visible to everyone; owners included unless added to the bypass list.

The PR routine (use it for every change from now on)

```
$ git switch main && git pull           # 1. start from the latest main
$ git switch -c <topic-branch>         # 2. e.g. ci/deploy-job
# ... edit files ...
$ git add -A && git commit -m "<message>" # 3. commit
$ git push -u origin <topic-branch>    # 4. publish the branch

# 5. GitHub: yellow banner 'Compare & pull request' → Create pull request
# 6. Wait for the checks → 'Squash and merge' → 'Delete branch'

$ git switch main && git pull           # 7. update local main
```

PR routine

i What CI does in this routine

- Step 4: no CI yet (push filter = main only).
- Step 5: CI runs on the PR - on a merge of your branch into the latest main.
- Step 6: after merging, CI runs again on main (push event) - Lab 5 deploys from that run.

Merge strategy

- Squash and merge = one tidy commit on main per PR. Most teams use it for feature branches.

✓ Keep this slide handy

- Steps 4.3 - 6.4 just say 'PR routine' and list what to change.

Step 4.3 Green PR: merge it (docs/owner branch)

LAB 4 • STEP 3 of 5

laptop terminal

```
$ git push -u origin docs/owner      # branch from Step 4.2 (README line)
# GitHub: Compare & pull request → Create pull request
```

✓ You should see on the PR page

- Checks section: 'Some checks haven't completed yet', then 5 green, e.g. 'CI / lint (pull_request)', 'CI / test (py3.11) (pull_request)' ... 'CI / build (pull_request)' - each marked Required.
- 'Checks' tab on the PR shows the same run (event: pull_request).
- Merge button turns green only after all required checks pass.
- Squash and merge → Delete branch. Actions tab: a new CI run on main (event push).
- Locally: git switch main && git pull → README has the Owner line.

! If it fails

- 'Required statuses must pass' with 'Expected - Waiting for status to be reported' → a check name in the ruleset is spelt differently from the job name. Edit the ruleset.

i Why is this the professional way?

- main is always green: nothing lands without passing CI.
- Every change has a PR: a place for review, discussion and history.
- The PR tests the MERGE result (refs/pull/N/merge), so it catches conflicts with the latest main.

Step 4.4 Red PR: watch the merge get blocked

laptop terminal

```
$ git switch main && git pull && git switch -c demo/red-pr
$ echo "import os" >> app/__init__.py
$ git commit -am "red PR demo" && git push -u origin demo/red-pr
# GitHub: Create pull request

# Fix it on the SAME branch (the PR updates itself):
$ git revert --no-edit HEAD && git push
# Then: Close pull request (nothing left to merge) + Delete branch
```

✓ You should see

- PR checks: CI / lint FAILED; test and build show as skipped.
- 'Merging is blocked' - the merge button is disabled (no bypass for you either).
- 'Files changed' tab shows the F401 annotation on app/__init__.py.
- After the revert push: a new run starts on the PR and goes green.

i Required check + skipped job

- Jobs skipped because a 'needs' failed count as not successful, so the PR stays blocked - exactly what we want.
- Fixing = pushing more commits to the branch; never 'force-merge'.

Step 4.5 See concurrency cancel an outdated run

LAB 4 • STEP 5 of 5

laptop terminal

```
$ git switch main && git pull && git switch -c demo/concurrency
$ echo "- note 1" >> README.md && git commit -am "note 1" && git push -u origin demo/concurrency
# GitHub: Create pull request, then IMMEDIATELY (within ~30 s):
$ echo "- note 2" >> README.md && git commit -am "note 2" && git push
```

✓ You should see (Actions → CI)

- The run for 'note 1' ends grey 'Cancelled' with an annotation like: Canceling since a higher priority waiting request for 'CI-refs/pull/<n>/merge' exists.
- The run for 'note 2' completes green.
- Merge the PR (squash) and delete the branch - or close it.

! Both runs finished?

- The first run was already done before the second push. Push note 3 faster, right after note 2.

i Why cancel?

- Only the newest commit matters for a PR - testing old commits wastes runner minutes and your waiting time.
- Never use cancel-in-progress on a deploy job you do not want interrupted halfway (use a separate group).
- Lab 4 done: compare with solution/lab4/ci.yml. Take a 10-minute break.

Lab 5 Deliver: protected environment, secret, approval, release

4 steps | 60 min | The wheel from the build job is deployed to 'staging' only after your approval, with a secret only that environment can read. A tag creates a GitHub Release.

Step 5.1 Create the 'staging' environment + secret

LAB 5 • STEP 1 of 4

```
$ python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

laptop terminal

1	Repo → Settings → Environments	New environment
2	Name	staging
3	Configure environment	
4	Tick: Required reviewers	add yourself → Save protection rules
5	Deployment branches and tags	Selected branches and tags → Add rule: main
6	Environment secrets → Add environment secret	Name: JWT_SECRET Value: <paste>

✓ You should see

- Settings → Environments lists staging with '1 secret' and protection rules; Required reviewers shows your name; the secret value is never shown again.

i Why an environment?

- The secret is released ONLY to jobs that declare environment: staging - not to every workflow and never to PR builds.
- Required reviewers = a human gate before deploying (like Jenkins input).
- Branch rule = only main can deploy to staging.
- Real teams add 'production' with stricter reviewers.

Step 5.2 Add the deploy-staging job (PR routine)

LAB 5 • STEP 2 of 4

```
deploy-staging:
  needs: build
  if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - uses: actions/checkout@v7          # scripts/ and seed/
    - uses: actions/setup-python@v7
      with:
        python-version: "3.13"
    - name: Download the wheel built by the build job
      uses: actions/download-artifact@v8
      with:
        name: expense-api-wheel
        path: dist
    - name: Deploy (start the app on this runner)
      env:
        JWT_SECRET: ${ secrets.JWT_SECRET }
      run: bash scripts/deploy_local.sh
    - name: Smoke test
      run: python scripts/smoke_test.py --base-url http://127.0.0.1:8000
    - name: Show application log
      if: always()
      run: cat "$HOME/staging/expense-api/app.log"
```

append to ci.yml

✓ You should see on the PR

- lint, 3x test, build green; deploy-staging shown as SKIPPED (the if: is false for pull_request).
- The PR is mergeable (deploy-staging is not a required check).

i Read the job

- download-artifact gets THE wheel built in this run - no rebuild.
- deploy_local.sh installs it into ~/staging/expense-api and starts python -m app in the background; it refuses to run without JWT_SECRET.
- 'Deploy' here = the runner itself (a stand-in for a real server) - the pattern is identical.

PR routine, branch ci/deploy-staging - do NOT merge yet

Step 5.3 Merge, approve the deployment, verify

LAB 5 · STEP 3 of 4

1	PR → Squash and merge	CI starts on main (push)
2	Actions → that CI run	after build: deploy-staging is 'Waiting'
3	Review deployments (yellow banner)	tick staging → Approve and deploy
4	Open job deploy-staging	read Deploy, Smoke test, Show application log

```
deploy-staging log (shortened)
>> Deploying dist/expense_api-1.0.0-py3-none-any.whl to /home/runner/staging/expense-api (port 8000)
[PASS] service is listening
[PASS] GET /expenses without token -> 401
[PASS] POST /auth/token -> 200 + token
[PASS] GET /expenses with token -> 200
SMOKE TEST PASSED
```

✓ You should see

- Before approval: 'Waiting for review: staging needs approval to start deploying changes.'
- After: deploy-staging green, SMOKE TEST PASSED; the env line in the log shows JWT_SECRET: *** (masked).
- Repo home page right column → 'Deployments' → staging (active), linked to this run.

! If it fails

- 'JWT_SECRET is not set' → secret created at repo level with another name, or job lacks environment: staging.
- Deploy job not waiting / no approval → required reviewers not saved.
- 'Branch main is not allowed to deploy to staging' → fix the deployment branch rule.
- Smoke FAIL → read 'Show application log' (config_error line).

Step 5.4 Tag a version: automatic GitHub Release

LAB 5 • STEP 4 of 4

.github/workflows/release.yml

```
name: Release
on:
  push:
    tags: ["v*.*.*"]
permissions:
  contents: write # needed to create a release
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.13"
      - name: Build wheel
        run: pip wheel . --no-deps -w dist
      - name: Create GitHub Release
        env:
          GH_TOKEN: ${github.token}
        run: |
          gh release create "$GITHUB_REF_NAME" dist/*.whl \
            --title "Expense API $GITHUB_REF_NAME" --generate-notes
```

! GITHUB_TOKEN

- Created per run, expires with it - no stored password. Tag pushes are allowed: the ruleset protects branches.

laptop terminal

```
# 1) PR routine: add release.yml (branch ci/release), merge
# 2) Tag the merged main and push the tag:
$ git switch main && git pull
$ git tag v1.0.0 && git push origin v1.0.0
```

✓ You should see

- Actions: 'Release' run triggered by tag v1.0.0, green.
- Repo → Releases: 'Expense API v1.0.0' with asset expense_api-1.0.0-py3-none-any.whl and auto-generated notes listing your merged PRs.

! If it fails

- 'Resource not accessible by integration' (HTTP 403) → permissions: contents: write missing. Re-run after the fix: delete the tag (git push --delete origin v1.0.0; git tag -d v1.0.0) and tag again.

Lab 6 Write it once: reusable workflows

4 steps | 45 min | Move lint + test matrix into a reusable workflow, call it from CI and from a nightly schedule, keep everything current with Dependabot.

Step 6.1 Create the reusable workflow

LAB 6 • STEP 1 of 4

python-quality.yml

```
name: Python quality (reusable)
on:
  workflow_call:           # = 'callable by other workflows'
  inputs:
    lint-targets:
      type: string
      default: "."
    python-versions:
      type: string
      default: '["3.13"]'    # JSON list
permissions:
  contents: read
jobs:
  lint:
    ...same steps as ci.yml lint, but:
    - name: Lint (ruff)
      env:
        LINT_TARGETS: ${ inputs.lint-targets }
      run: ruff check $LINT_TARGETS --output-format=github
  test:
    needs: lint
    strategy:
      fail-fast: false
    matrix:
      python-version: ${ fromJSON(inputs.python-versions) }
    ...same steps as ci.yml test
```

i How it works

- workflow_call makes the whole file a 'function' with typed inputs.
- Inputs are strings, numbers or booleans - a list is passed as JSON text and turned back into a list with fromJSON().
- Inputs reach the shell through env: (script-injection rule again).
- Callers: same repo `./.github/workflows/x.yml`, or another repo owner/repo `./.github/workflows/x.yml@v1`.

Do

- Copy solution/lab6/python-quality.yml into `./.github/workflows/` on a new branch `ci/reusable` (do not push yet - next step).

Step 6.2 Call it from CI + update the ruleset

LAB 6 • STEP 2 of 4

```
ci.yml
jobs:
  quality:                                # replaces the lint and test jobs
    uses: ../github/workflows/python-quality.yml
    with:
      lint-targets: "app tests scripts"
      python-versions: '["3.11", "3.12", "3.13"]'

  build:
    needs: quality                        # was: needs: test
    ...unchanged...
  deploy-staging:
    ...unchanged...
```

✓ You should see on the PR (branch ci/reusable)

- New check names: 'CI / quality / lint (pull_request)', 'CI / quality / test (py3.11) (pull_request)' ... all green.
- Old required names (lint, test (py3.x)) show 'Expected - Waiting for status to be reported' → merge blocked!
- Fix: Settings → Rules → protect-main → remove the 4 old checks, add quality / lint and quality / test (py3.11/3.12/3.13) → Save. The PR becomes mergeable.
- Merge → main run: quality → build → deploy-staging (approve) green.

i A real-world lesson

- Required check names are a contract. Renaming jobs or moving them into a reusable workflow changes the names.
- Teams update the ruleset in the same change window - otherwise every open PR is stuck.
- Win: ci.yml shrank, and any other workflow can now run the same quality gate with 4 lines.

Step 6.3 STRETCH: nightly schedule + choice input

.github/workflows/nightly.yml

```
name: Nightly
on:
  schedule:
    - cron: "30 20 * * *"          # 20:30 UTC = 02:00 IST
  workflow_dispatch:
  inputs:
    python-versions:
      description: "Which Pythons?"
      type: choice
      options: ['"3.13"', '"3.11"', '"3.12"', '"3.13"']
      default: '"3.11"', '"3.12"', '"3.13"'
permissions:
  contents: read
jobs:
  quality:
    uses: ../github/workflows/python-quality.yml
    with:
      lint-targets: "app tests scripts"
      python-versions: ${ inputs.python-versions || '"3.11"', '"3.12"', '"3.13"' }
```

PR routine (branch ci/nightly) → merge

✓ Verify now (manual)

- Actions → Nightly → Run workflow: a dropdown 'Which Pythons?' with 2 choices.
- Choose ["3.13"]: the run has quality / lint + ONE test job.

✓ Verify tomorrow (schedule)

- Actions → Nightly shows a run marked 'Scheduled' around 02:00-02:30 IST (scheduled runs can start some minutes late).

i cron is UTC

- Always convert from IST (UTC+5:30). Schedules run on the default branch only.

Step 6.4 STRETCH: Dependabot keeps versions current

LAB 6 • STEP 4 of 4

`.github/dependabot.yml`

```
# .github/dependabot.yml    (next to workflows/, NOT inside it)
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
```

PR routine (branch ci/dependabot) → merge

✓ You should see

- Repo → Insights → Dependency graph → Dependabot tab lists github-actions and pip for '/', each with a 'Last checked' time.
- If a newer version of a pinned package or action exists, Dependabot opens a PR - and your CI checks it automatically.

i Why it matters

- Pinned versions (ruff==0.16.10, @v7) are safe but go stale.
- Dependabot + required CI = upgrades proposed AND tested, you just review and merge.
- Lab 6 done: compare with solution/lab6/.

Security checklist for every workflow you write

1

Least privilege

permissions: contents: read at the top; raise per workflow/job only when needed (release: contents: write).

2

Secrets in environments

Deploy secrets in protected environments, never in YAML, never echoed (they are masked, but do not rely on it).

3

No script injection

Never `${{ inputs / PR title / branch name }}` inside run: - pass via env: and use `"$VAR"`.

4

Pin and update actions

Trusted owners, major tag or SHA pin, Dependabot for updates.

5

Protect main

Ruleset: PR required + required checks + no force pushes.

6

Forks are untrusted

`pull_request` from forks runs without secrets - keep it that way; avoid `pull_request_target` unless you know exactly why.

Final checkpoint: prove it to yourself

- ☐ Hello Actions ran by push AND by button with my input.
- ☐ I saw an annotation on the exact line and downloaded a test report.
- ☐ CI graph: quality (lint + 3 tests) → build → deploy-staging.
- ☐ A red PR was blocked; a green PR merged via the PR routine.
- ☐ I approved a deployment; smoke test passed with a masked secret.
- ☐ Release v1.0.0 exists with the wheel attached.
- ☐ I updated the ruleset after the check names changed.

Quick quiz (answers in speaker notes)

- Q1. Why does every job start with actions/checkout?
- Q2. How do two jobs share a file?
- Q3. Why quote "3.10" in YAML?
- Q4. A PR from a fork needs JWT_SECRET for its tests. What happens and why?
- Q5. You renamed job 'lint' to 'ruff'. What breaks?
- Q6. Where do you put the approval gate for production?

Clean-up and next steps

Clean-up (pick one)

- Keep the repo: Actions → Nightly → '...' → Disable workflow (stops the schedule).
- Remove all: Settings → General → Danger Zone → Delete this repository.
- Delete the token you created in Step 1.1 if you no longer need it.

✓ Check

- Actions → Nightly shows 'This workflow was disabled manually', or the repo URL returns 404.

i Practise on your own

- Add a 'production' environment with a second reviewer and a deploy-production job after staging.
- Add runs-on: [ubuntu-latest, windows-latest] to the matrix (fix the Windows-only problems you find).
- Turn python-quality.yml into a cross-repo reusable workflow and call it from a 2nd repo with @v1.
- Publish the coverage report (pytest-cov) to the job summary.
- Read: docs.github.com/actions → 'Security hardening for GitHub Actions'.

Appendix A Troubleshooting: symptom → fix

Symptom	Most likely cause	Fix
No run appears after push	File not in .github/workflows/ or branch filter	Check folder name (dot!) and on: branches
refusing to allow ... workflow	Token lacks 'Workflows' permission	New fine-grained token with Workflows: read/write
Invalid workflow file ... line N	YAML indentation / tab / missing space	Fix line N; use 2 spaces; actionlint locally
No 'Run workflow' button	No workflow_dispatch or not on main	Add trigger, merge to main
ruff/pytest: command not found	Install step missing / wrong file	pip install -r requirements-dev.txt
Artifact: No files were found	Path typo or step before it failed	Same path as --junitxml; if: always()
PR waits 'Expected - Waiting for status'	Required check name not produced	Edit ruleset check names (Lab 6!)
GH013 rule violations on push	Ruleset protects main	Use the PR routine
JWT_SECRET is not set	Secret name/env level / no environment:	Environment staging secret JWT_SECRET
Resource not accessible by integration	GITHUB_TOKEN permission too low	permissions: contents: write (release)

Appendix B Cheat sheet

workflow syntax

```
on:
  push: { branches: [main], tags: ["v*.*.*"] }
  pull_request: { branches: [main] }
  workflow_dispatch: { inputs: { x: { type: string } } }
  schedule: [ { cron: "30 20 * * *" } ] # UTC
  workflow_call: { inputs: { y: { type: string } } }
permissions: { contents: read }
concurrency: { group: "${{ github.workflow }}-${{ github.ref }}",
              cancel-in-progress: true }
jobs:
  a:
    runs-on: ubuntu-latest
    strategy: { fail-fast: false, matrix: { v: ["3.12", "3.13"] } }
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
      - with: { python-version: "${{ matrix.v }}", cache: pip }
      - run: pytest -q
      - if: always()
      - uses: actions/upload-artifact@v7
      - with: { name: "r-${{ matrix.v }}", path: reports/ }
  b:
    needs: a
    if: github.ref == 'refs/heads/main'
    environment: staging
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v8
      - env: { S: "${{ secrets.S }}" }
      - run: ./deploy.sh
```

tools

```
# Lint workflows locally (pip)
pip install actionlint-py
actionlint

# GitHub CLI
gh run list
gh run watch
gh run view --log-failed
gh workflow run hello.yml \
  -f greeting="Asha"

# Useful env vars in run:
$GITHUB_SHA $GITHUB_REF_NAME
$GITHUB_RUN_NUMBER $RUNNER_OS
$GITHUB_STEP_SUMMARY (append Markdown)
```

Appendix C Trainer notes: run this class smoothly

Day before (trainer)

- Do the full lab once with a fresh GitHub account on the FIL network: pushes, Actions, environment approval, release.
- Confirm learners can reach github.com and push (proxy, SSO, token policy). Corporate GitHub orgs may restrict Actions or public repos - use personal accounts as planned.
- Share the kit zip and the laptop checklist slide in advance.

During class

- Project the Actions tab: every run is a live demo. Pause on graphs, annotations, artifacts.
- Most time lost on: token/workflow scope (1.1), YAML indentation (1.2/2.1), ruleset check names (4.2/6.2), environment secret level (5.1).
- While runs execute (30 s - 2 min), discuss the WHY box.
- Everyone's runs are independent - no shared server to overload.

✓ What was verified when this kit was built

- Every workflow in solution/ passed actionlint 1.7.12 (with the reusable workflow wired up, as in the real repo).
- All commands inside the workflows executed locally: ruff + 13 tests on Python 3.11, 3.12 and 3.13; the batched test fails only on 3.11; wheel build; deploy_local.sh + smoke test 4/4 PASS; missing-secret error; YAML break detected.
- GitHub UI screens and hosted runs could not be executed from the build sandbox: do one dry run before class.